

Measurement Simulation and State Estimation

Jett Vanbrocklin

March 2026

1 Introduction

State estimation for a quadrotor relies on a consistent description of its translational dynamics, rotational motion, and the way onboard sensors observe the vehicle state, as outlined in [1]. In this laboratory assignment, these relationships are translated into a practical simulation and estimation framework. The work centers on implementing a discrete-time dynamics model for the quadrotor and detailed sensor models for the IMU, GNSS, and onboard camera, along with an attitude reconstruction algorithm based on Wahba's problem. Together, these elements provide noisy but physically consistent measurements and state propagation, forming the basis for testing and evaluating a navigation filter. The report describes the relevant theory, the specific modeling and implementation choices, and the resulting estimation performance.

2 Theoretical Analysis

2.1 Proving the Rank of the Baseline Measurement Covariance Matrix

The baseline vector measurement model uses the known distances of the two antennas in the drone to find $\tilde{\mathbf{r}}_{bG}$. Due to the lack of error in the *length* of $\tilde{\mathbf{r}}_{bG}$, the baseline measurement covariance matrix $\mathbf{R}_{bG}$ is formed with

$$\mathbf{R}_{bG} = \|\mathbf{r}_{bG}\|^2 \sigma_b^2 [\mathbf{I}_{3 \times 3} - \mathbf{r}_{bG}^u (\mathbf{r}_{bG}^u)^T] \quad (1)$$

where $\mathbf{r}_{bG}^u = \mathbf{r}_{bG} / \|\mathbf{r}_{bG}\|$ and σ_b is the standard deviation of the angular error. To prove that $\mathbf{R}_{bG}$ has rank 2, let's first simplify the expression into an arbitrary matrix. The coefficient values $\|\mathbf{r}_{bG}\|^2$ and σ_b^2 can be set to 1, as they have no effect on the rank.

$$\begin{aligned} \text{rank}(\mathbf{R}_{bG}) &= \text{rank}(\|\mathbf{r}_{bG}\|^2 \sigma_b^2 [\mathbf{I}_{3 \times 3} - \mathbf{r}_{bG}^u (\mathbf{r}_{bG}^u)^T]) \\ &= \text{rank}([\mathbf{I}_{3 \times 3} - \mathbf{r}_{bG}^u (\mathbf{r}_{bG}^u)^T]) \end{aligned}$$

Now, the expression can be described as the difference between the 3 x 3 identity matrix and the outer product of a unit vector. Let this expression be denoted by $\mathbf{P}$ where $P = [I_{3 \times 3} - r_{bG}^u (r_{bG}^u)^T]$. The goal is to prove that $\text{rank}(P) = 2$. To start, consider that P is a 3 x 3 matrix. Then,

$$\text{rank}(P) + \dim(\ker(P)) = 3 \quad (2)$$

Therefore, the rank of P can be determined by finding $\dim(\ker(P))$. We can first show that r_{bG}^u is in the kernel of P .

$$Pr_{bG}^u = (I_{3 \times 3} - r_{bG}^u (r_{bG}^u)^T) r_{bG}^u = I_{3 \times 3} r_{bG}^u - r_{bG}^u (r_{bG}^u)^T r_{bG}^u = r_{bG}^u - r_{bG}^u (r_{bG}^u)^T r_{bG}^u$$

But $(r_{bG}^u)^T r_{bG}^u = 1$, so

$$Pr_{bG}^u = r_{bG}^u - r_{bG}^u (r_{bG}^u)^T r_{bG}^u = r_{bG}^u - r_{bG}^u = 0$$

Therefore,

$$r_{bG}^u \in \ker(P)$$

So, $\dim(\ker(P)) \geq 1$. Now, let's show that there are no other elements in $\ker(P)$. First, let $x \in R^3$

$$Px = 0$$

Then,

$$(I_{3 \times 3} - r_{bG}^u (r_{bG}^u)^T)x = 0$$

Rewriting,

$$x - r_{bG}^u (r_{bG}^u)^T x = 0 \longrightarrow x = r_{bG}^u (r_{bG}^u)^T x$$

Now, $(r_{bG}^u)^T x$ is a scalar, denote it by α :

$$\alpha = (r_{bG}^u)^T x$$

Then, rewriting once more

$$x = r_{bG}^u \alpha = \alpha r_{bG}^u$$

Thus, any vector in the kernel of P must be a scalar multiple of r_{bG}^u . Therefore,

$$\ker(P) = \text{span}(r_{bG}^u)$$

and in turn

$$\dim(\ker(P)) = 1$$

Finally, since P is a 3×3 matrix, we can plug back into (2):

$$\begin{aligned} \text{rank}(P) + \dim(\ker(P)) &= 3 \\ \text{rank}(P) + 1 &= 3 \\ \text{rank}(P) &= 2 \end{aligned}$$

By the earlier simplification, rank P is transitive to rank R_{bG} , and hence

$$\text{rank}(R_{bG}) = 2$$

That concludes our proof that the baseline measurement covariance matrix has a rank of 2.

2.2 Intersection of Two Lines

A line defined by

$$ax + by + c = 0$$

can be represented by $\mathbf{l} = [a, b, c]^T$. I'd like to show that the point defined as the intersection of two lines $\mathbf{l}_1$ and $\mathbf{l}_2$ can be found as $\mathbf{x} = \mathbf{l}_1 \times \mathbf{l}_2$.

Consider $\mathbf{x}$ can be found on both $\mathbf{l}_1$ and $\mathbf{l}_2$. Then,

$$l_1^T \mathbf{x} = 0, \quad l_2^T \mathbf{x} = 0$$

by the definition of line stated previously. Now let $\mathbf{x} = \mathbf{l}_1 \times \mathbf{l}_2$. Then,

$$\begin{aligned} l_1^T \mathbf{x} &= l_1^T (\mathbf{l}_1 \times \mathbf{l}_2) = 0 \\ l_2^T \mathbf{x} &= l_2^T (\mathbf{l}_1 \times \mathbf{l}_2) = 0 \end{aligned}$$

by the property of the cross product. These support the previously stated equations that are incidental to $\mathbf{x}$ lying on both $\mathbf{l}_1$ and $\mathbf{l}_2$. Therefore, the point defined by the intersection of two lines $\mathbf{l}_1$ and $\mathbf{l}_2$ can be found as $\mathbf{x} = \mathbf{l}_1 \times \mathbf{l}_2$.

2.3 Acceleration in I frame of a point in B frame

The goal of this section is to prove the following:

$$a_I = \ddot{R}_I + R_{BI}^T [\omega_B \times (\omega_B \times l_B) + (\dot{\omega}_B \times l_B)] \quad (3)$$

First, let's denote the position of the point in the inertial frame to be

$$r_I = R_I(t) + R_B^T l_B$$

using frame translation notation. Then, by taking the first derivative over time of r_I , we can obtain velocity in the inertial frame.

$$\begin{aligned} \frac{dr_I}{dt} &= \dot{R}_I(t) + \frac{d}{dt}(R_B^T l_B) \\ &= \dot{R}_I(t) + R_B^T [\omega_B \times] l_B \\ &= \dot{R}_I(t) + R_B^T (\omega_B \times l_B) \end{aligned}$$

where the rotation of frame B has angular velocity ω_B . Then, by taking the derivative once more,

$$\begin{aligned} \ddot{r}_I = a_I &= \frac{d}{dt}(\dot{R}_I(t) + R_B^T (\omega_B \times l_B)) \\ &= \frac{d}{dt}\dot{R}_I(t) + \frac{d}{dt}(R_B^T (\omega_B \times l_B)) \\ &= \ddot{R}_I(t) + (R_B^T [\omega_B \times] (\omega_B \times l_B) + R_B^T (\dot{\omega}_B \times l_B)) \\ &= \ddot{R}_I(t) + R_B^T (\omega_B \times (\omega_B \times l_B)) + R_B^T (\dot{\omega}_B \times l_B) \\ &= \ddot{R}_I(t) + R_B^T (\omega_B \times (\omega_B \times l_B) + (\dot{\omega}_B \times l_B)) \end{aligned}$$

by product rule and algebraic simplification.

Therefore, we have shown that

$$a_I = \ddot{R}_I(t) + R_B^T (\omega_B \times (\omega_B \times l_B) + (\dot{\omega}_B \times l_B))$$

2.4 Lever Arm Exclusion in the Rate Gyro Measurement

The model for the rate gyro measurement is

$$\tilde{\omega}_B = \omega_B + b_g + v_g \quad (4)$$

where b_g is the gyro bias and v_g is noise. It can be seen that the lever arm l_B is not considered in the model for the rate gyro.

This is because the gyro model describes the measurement of angular velocity, which is uniform across a rigid body. In other words, it does not depend on

where the gyro is located on the body. A lever arm l_B only affects the linear velocity or acceleration of an offset point, but angular velocity is unaffected. Since both the gyro and the accelerometers are in the same IMU, the lever arm is necessary for modeling the accelerometer, yielding the $\omega \times (\omega \times l_B)$ and $\dot{\omega} \times l_B$ terms, but irrelevant for the angular rate equation.

3 Implementation

3.1 GNSS Simulator

The `gnssMeasSimulator` function computes GNSS measurements for a quadrotor's two antennas based on its state and sensor parameters. Using the quad's inertial position $\mathbf{r}_I$ and attitude $\mathbf{R}_{BI}$, it transforms the known primary and secondary antenna offsets from the body frame into the inertial frame, then into the GNSS/ECEF frame using a reference rotation from `Recef2enu`. From these, it outputs the primary antenna position in GNSS coordinates ($rpGtilde$) and the baseline vector from the primary to the secondary antenna ($rbGtilde$). The code also sets up covariance matrices and Gaussian noise generation for both the primary position and baseline.

3.2 HD Camera Simulator

The `hdCameraSimulator` function simulates the 2D pixel location of a 3D feature point as seen by the quad's high-definition camera. Given the feature position in the inertial frame $\mathbf{r}_{XI}$, the quad's position $\mathbf{r}_I$ and attitude $\mathbf{R}_{BI}$, and camera parameters from `sensorParams`, it first constructs the camera orientation relative to the inertial frame as and computes the camera translation term. It then forms a homogeneous 3D point and projects it onto the image plane using the camera intrinsic matrix $\mathbf{K}$ via a standard pinhole camera model. The resulting homogeneous image coordinates are normalized to obtain pixel coordinates, and zero-mean Gaussian noise with covariance Rc is added to model measurement noise. The function returns this noisy 2D measurement $\mathbf{rx}$ in pixels. The function finally checks the projected coordinates in reference to the bounds to determine whether the feature point is in frame (visible).

3.3 IMU Simulator

The `imuSimulator` function generates simulated IMU measurements for the quadrotor's accelerometers and gyros, including realistic noise and bias dynamics. Using the quad's current inertial-frame acceleration $\mathbf{a}_I$, attitude $\mathbf{R}_{BI}$, and body angular rate $\boldsymbol{\omega}_B$ from `S.statek`, together with sensor and constant parameters in $\mathbf{P}$, it computes the specific force in the body frame as the gravity-compensated acceleration transformed into the body frame and then corrupts it with accelerometer bias and noise. The accelerometer bias $\mathbf{b}_a$ is modeled as a first-order Gauss–Markov process, initialized in steady state and stored in a persistent variable so it evolves smoothly over time. A similar Gauss–Markov

model is used for the gyro bias $\mathbf{b}_g$. White measurement noise with covariances $\mathbf{Q}_a$ (accelerometer) and $\mathbf{Q}_g$ (gyro) is added to the biased signals, resulting in the simulated specific force $\mathbf{f}_{\text{tilde}}B$ and angular rate $\boldsymbol{\omega}_{\text{tilde}}B$ that mimic realistic IMU outputs.

3.4 Wahba’s Solver

The `wahbaSolver` function computes the optimal rotation matrix $\mathbf{R}_{BI}$ that aligns a set of inertial-frame unit vectors with corresponding body-frame unit vectors in a weighted least-squares sense. The function directly follows the methodology list in Section 7.4 of (1), using Singular Value Decomposition.

3.5 Measurement Model

The `h_meas` function implements the measurement model used by the quad’s estimator, mapping the state vector and measurement noise into a stacked measurement vector. Using sensor parameters (antenna locations and camera pose), it computes the inertial-frame position of the primary GNSS antenna $\mathbf{r}_{pI}$ and the unit vector from the primary to the secondary antenna $\mathbf{r}_{bI}$. It also determines the camera center position in the inertial frame and, for each feature flagged as visible by `mcVeck`, forms the unit vector from the camera to that feature, expressed in the camera frame. These quantities are stacked into the nominal measurement vector.

3.6 Dynamics Model

The `f_dynamics` function propagates the quad’s state vector forward one IMU time step using a discrete-time process model driven by IMU measurements and process noise. The measured body rates and specific force are corrected for gyro and accelerometer biases and white noise to obtain “true” angular rate and acceleration, from which Euler angle rates and inertial-frame acceleration are computed. The bias for the gyro and accelerometer model are also propagated through each iteration.

4 Experimentation

4.1 Testing Wahba’s Solver

To test the `wahbaSolver` function, I first used a very simple identity test. I set the true rotation matrix to $\mathbf{I}$ and used three orthogonal unit vectors as inertial and body vectors ($\mathbf{v}_1 = [1, 0, 0]^T$, $\mathbf{v}_2 = [0, 1, 0]^T$, $\mathbf{v}_3 = [0, 0, 1]^T$). Since the body and inertial vectors were identical, the correct solution is the identity matrix:

$$R_{BI} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

I then used a 90° rotation about the z-axis with inertial vectors $v_{1I} = [1, 0, 0]^T$ and $v_{2I} = [0, 1, 0]^T$ (unit x and y axes). The body-frame vectors were created analytically using the known rotation $v_{iI} = R_{trueBI}v_i$ where.

$$R_{trueBI} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

The recovered rotation matrix from wahbaSolver function matched the true rotation with very small error.

These two tests are very simple to construct and interpret, yet they are strong enough to reveal common implementation errors such as swapped frames, incorrect transposes in forming the matrix $\mathbf{B}$ and sign mistakes in the SVD-based reconstruction.

4.2 Circular Path with Estimation

Integrating the sensor simulation models, the measurement model, and the dynamics model described above, the parameters that yield a circular trajectory for the quad were fed into the model. **Figure 1** and **Figure 2** depict the true horizontal and vertical positions of the quad when simulating the circular path. Even though these models introduce noise into the system, the path still follows a circular path to a high degree of accuracy.

The vertical trajectory can be seen to follow a common PD oscillation converging at 1 meter. The oscillation continues around the 1 meter mark due to the introduced noise.

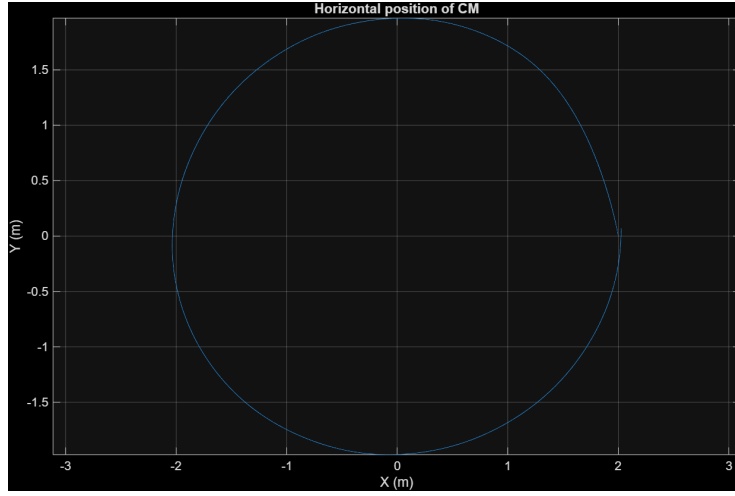


Figure 1: True Horizontal Position of Quad

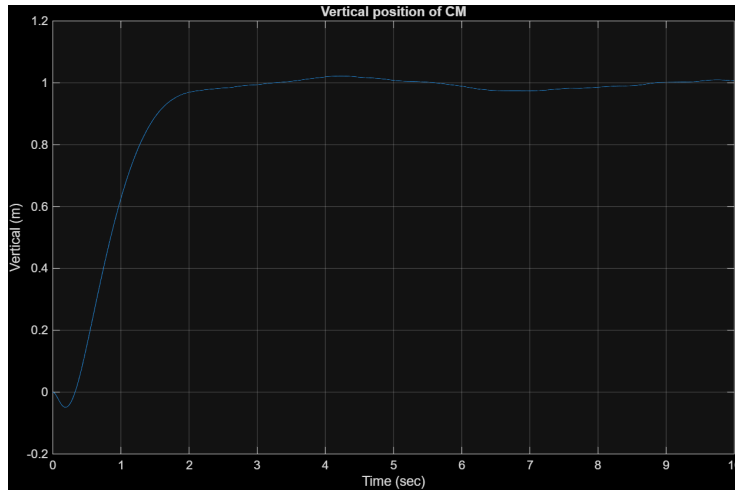


Figure 2: True Vertical Position of Quad

4.3 Comparing Estimated and Non-Estimated Paths

In this section, I would like to compare the trajectory performance of the estimated vs the non-estimated models.

To carry out this comparison, I will be relating the Yaw Error Angle over time for the two models, as well as looking at the variation of the Yaw Error Angle over numerous simulations of the estimated model (this is because the noise is random).

First, Let us look at single outputs of each model. **Figure 3** depicts the yaw error angle. for the estimated model and **Figure 3** depicts the yaw error for the non-estimated model. As shown in these figures, the non-estimated model has smoother convergence to a steady value than the estimated model. This is to be expected, as the ideal model is perfectly responsive and includes zero noise. The estimated model is as the name suggest, an estimation based on noisy data and noisy measurement. However, even through the estimated model, convergence is clear and quite responsive. Next, I would like to compare the various cases of the estimated model displayed in **Figure 5, 6, and 7**.

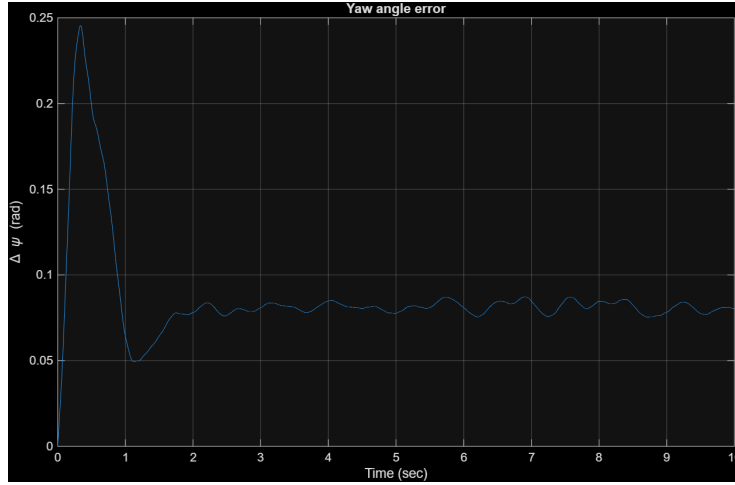


Figure 3: Estimated Model Yaw Angle Error

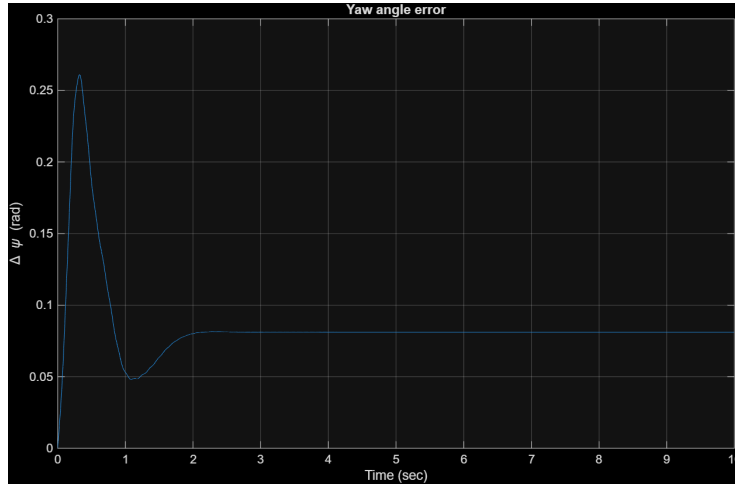


Figure 4: Non-Estimated Model Yaw Angle Error

The following figures (**Figure 5, 6, 7**) display the variation in the estimated yaw angle error.

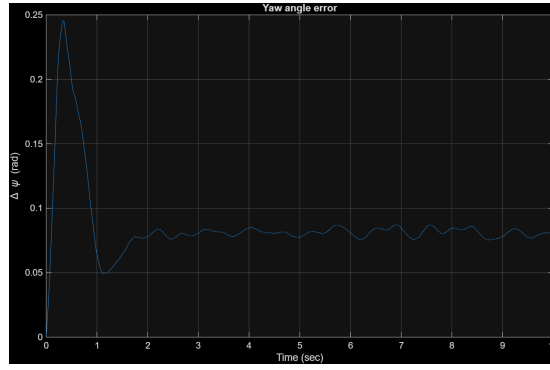


Figure 5: Estimated Model Yaw Angle Error Case 1

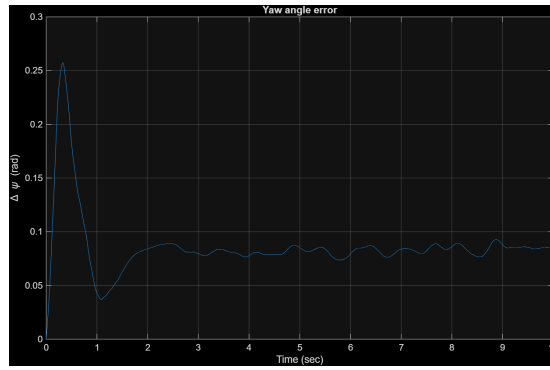


Figure 6: Estimated Model Yaw Angle Error Case 2

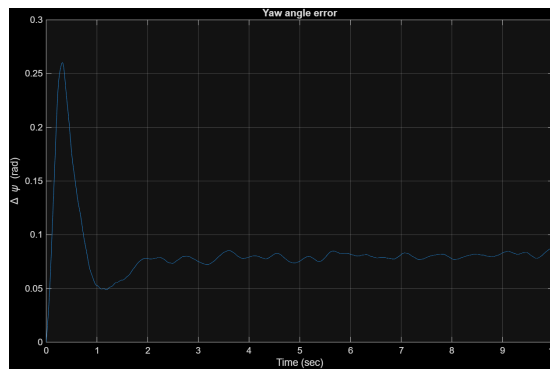


Figure 7: Estimated Model Yaw Angle Error Case 3

The yaw angle error is quite variable in locations of local extrema. The plots are consistently converging at with similar response time. These results are also expected, as the noise introduced in the system is random, but the effects of a consistent controller are clear through the random small fluctuations in error.

4.4 Effects of Camera

By removing the coordinates of feature points in the simulation environment, one could disable the effects of the HD camera simulator (Section 3.2 Above). I would now like to analyze the effects (or lack thereof) of this camera on the model. In **Figure 8, 9, and 10**, the horizontal trajectory, vertical trajectory, and yaw error angle respectfully are shown as output to the simulation without the inclusion of the camera model. Based on these plots, it is fair to conclude the state estimates are sufficiently accurate for quad control. Next, to compare the performance of the model in reference to the simulation including the camera, I will once again use the yaw error angle. In **Figure 3**, the model using the camera can be seen to have a slightly less stable yaw error than the camera-less model. However, the convergence for the yaw angle error appears to be as responsive with and without the camera, and the level of convergence also appears to be consistent. This Is an expected result of the given test. Only a single feature point is provided for the camera to respond to. Noise is also introduced in the camera model, which could mean the information received and processed by the camera model is not enough to make a great difference in the response of the model, causing the similar results between camera and camera-less cases.

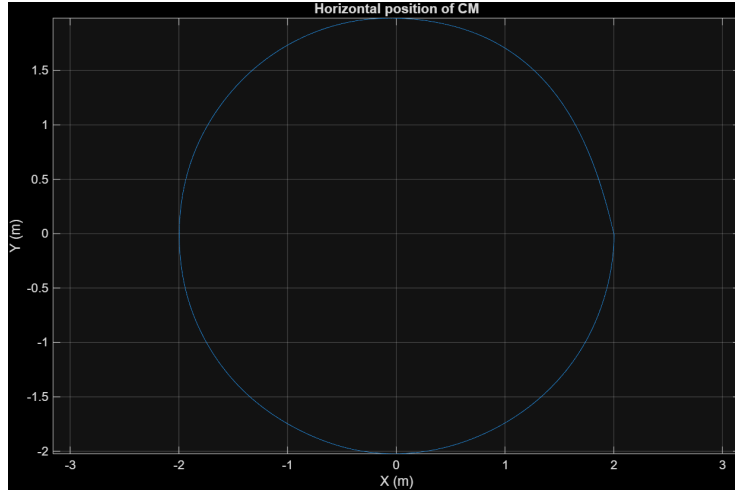


Figure 8: Horizontal Trajectory without Camera Model

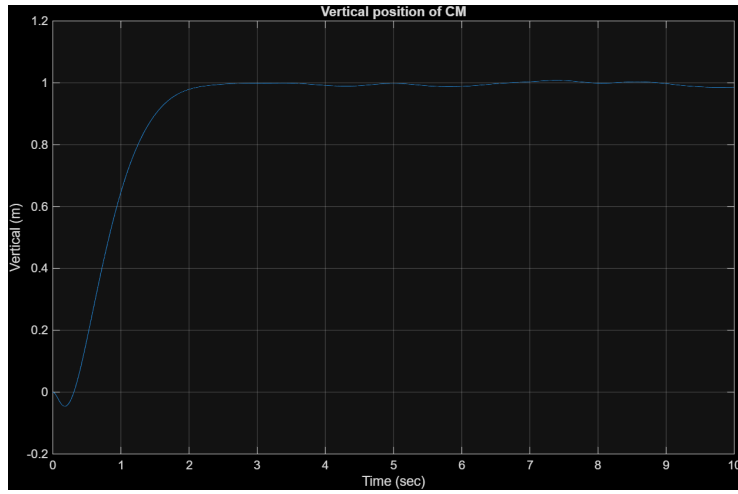


Figure 9: Vertical Trajectory without Camera Model

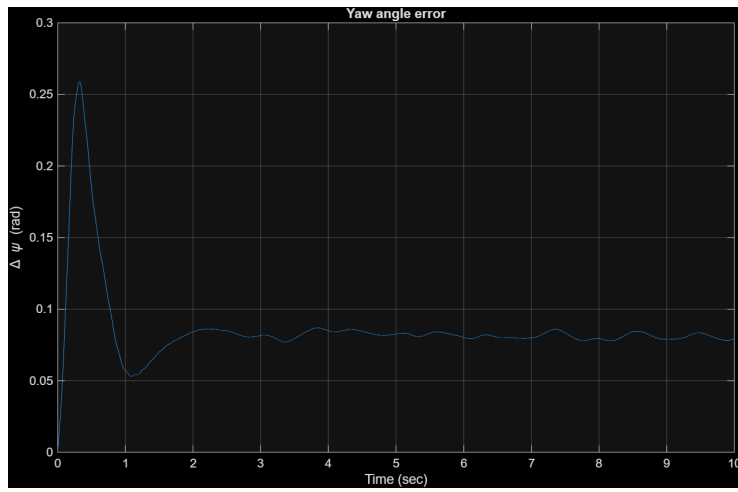


Figure 10: Yaw Angle Error without Camera Model

References

- 1 Humphreys, Todd. "Main." Jan. 2026.
- 2 Vanbrocklin, Jett. "Simulating Quadrotor Dynamics." Feb. 2026.
- 3 Humphreys, Todd. "lab3." Feb. 2026.